DISTRIBUTED MACHINE LEARNING K-MEANS CLUSTERING VIA PARAMETER SERVER

RAMAHLO RS
202391068

RAMPHADI KM
240036726

RAKGETSE PI
202204262

RAMOLOTO CB
240000760

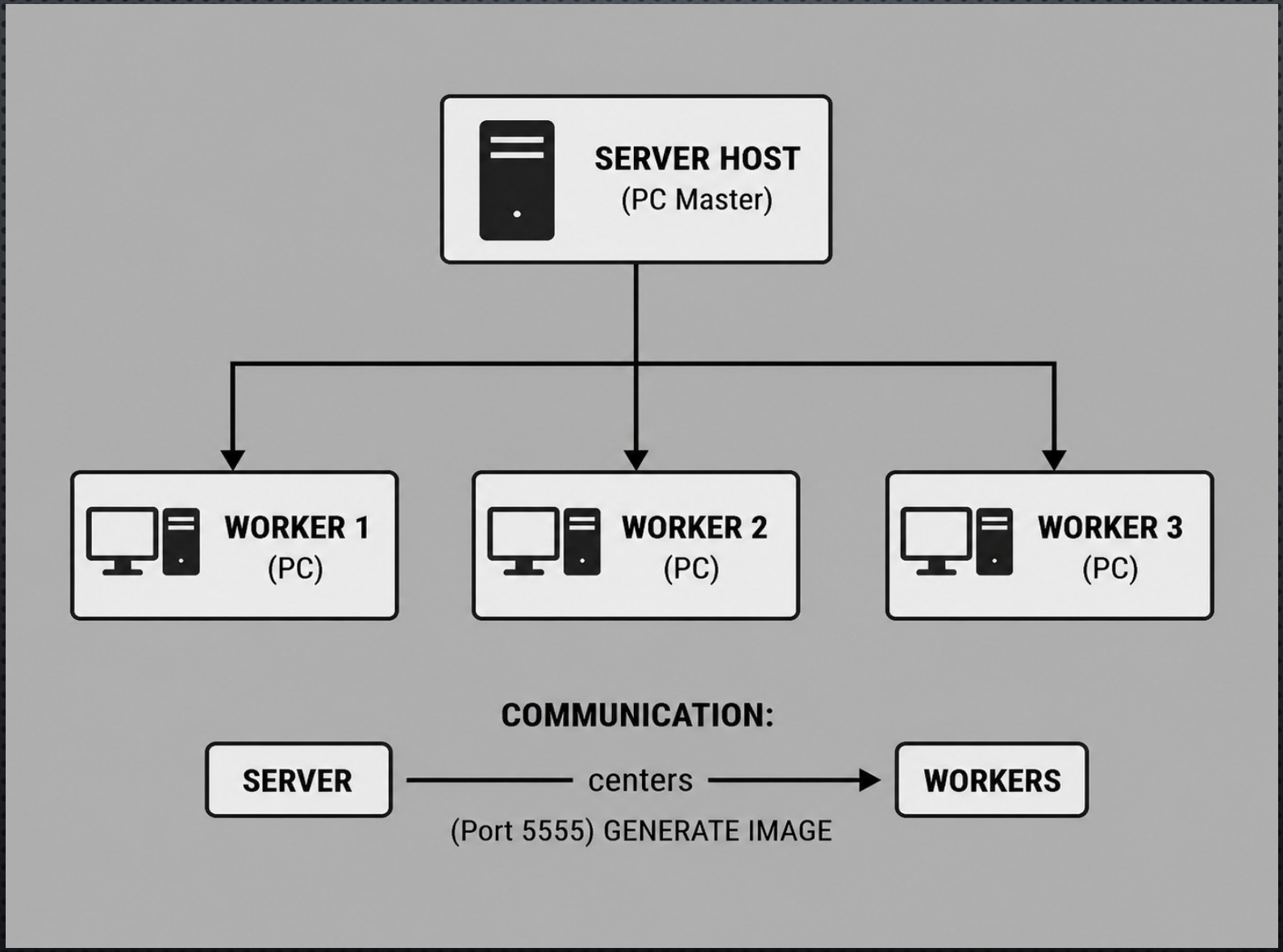
RADEBE K
202176691

GROUP M

Introduction

- Problem Statement: Single machines struggle with massive datasets that exceed RAM or compute limits.
- Motivation: Distributed ML splits workload across multiple nodes to scale training.
- Objective: Implement distributed K-Means clustering across 3 laptops.
- Dataset: Iris dataset augmented to 150,000 samples.

SYSTEM ARCHITECTURE



Architecture continued

- Wi-Fi Hotspot for isolated communication.
- Software: Python, ZeroMQ (messaging), NumPy (math), Pandas (data handling).
- Architecture: Laptop 1 (Parameter Server), Laptops 2 & 3 (Worker Nodes).

METHODOLOGY

- DATASET:IRIS DATASET
- ALGORITHM: K-MEANS CLUSTERING WITH PARAMETER SERVER SYNCHRONIZATION.
- DATA PARTITIONING: `IRIS_AUGMENTED.CSV` (150K ROWS) SPLIT 50/50 VIA HASH-BASED INDEXING.
- WORKERS RECEIVE UNIQUE,NON-OVERLAPPING DATA SUBSETS

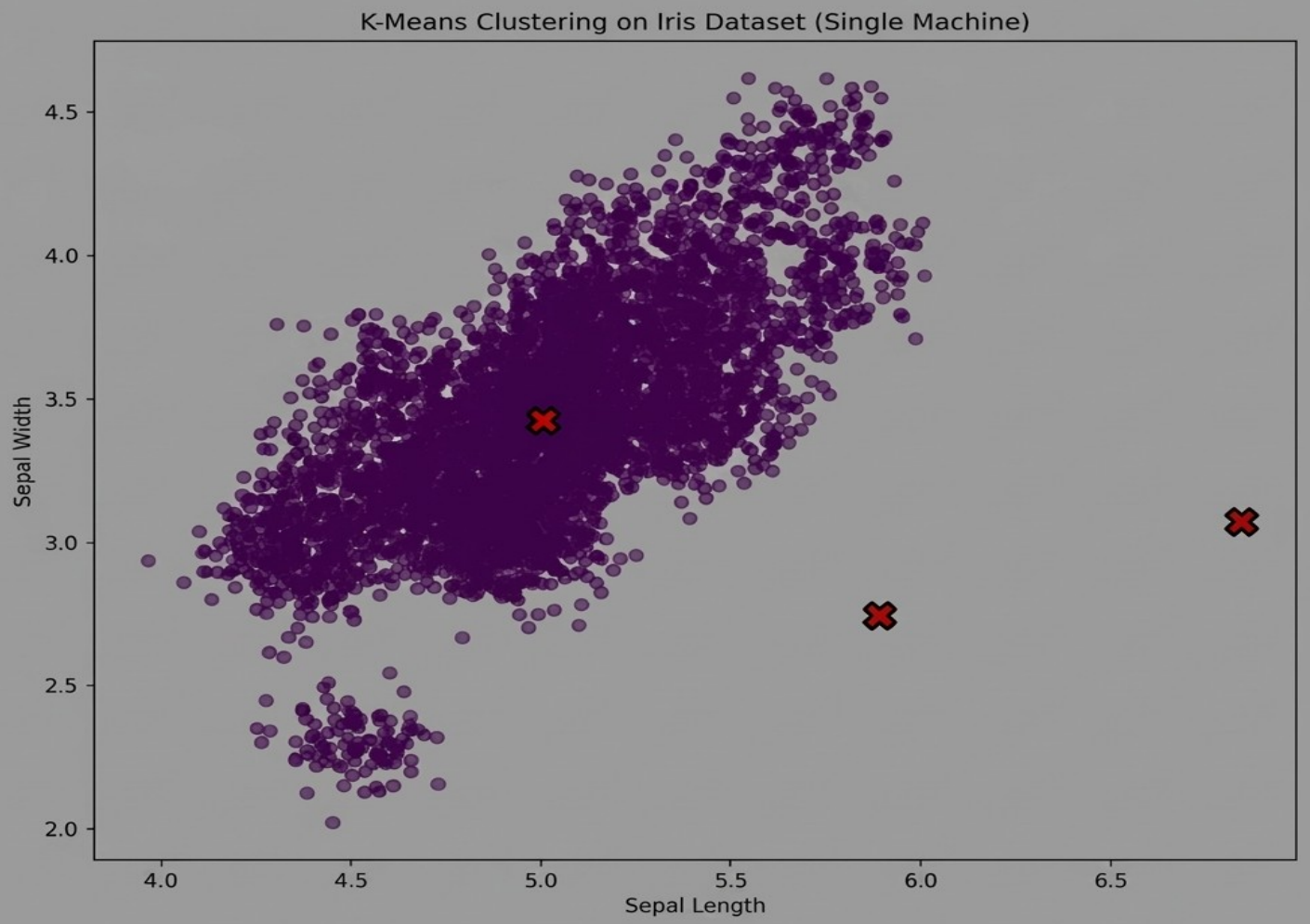
Methodology continued

- Workflow:
  1. Server broadcasts global cluster centers.
  2. Workers compute local cluster means on their data partition.
  3. Workers send weighted updates back to the server.
- Synchronization: Server averages updates based on sample count (weighted mean).

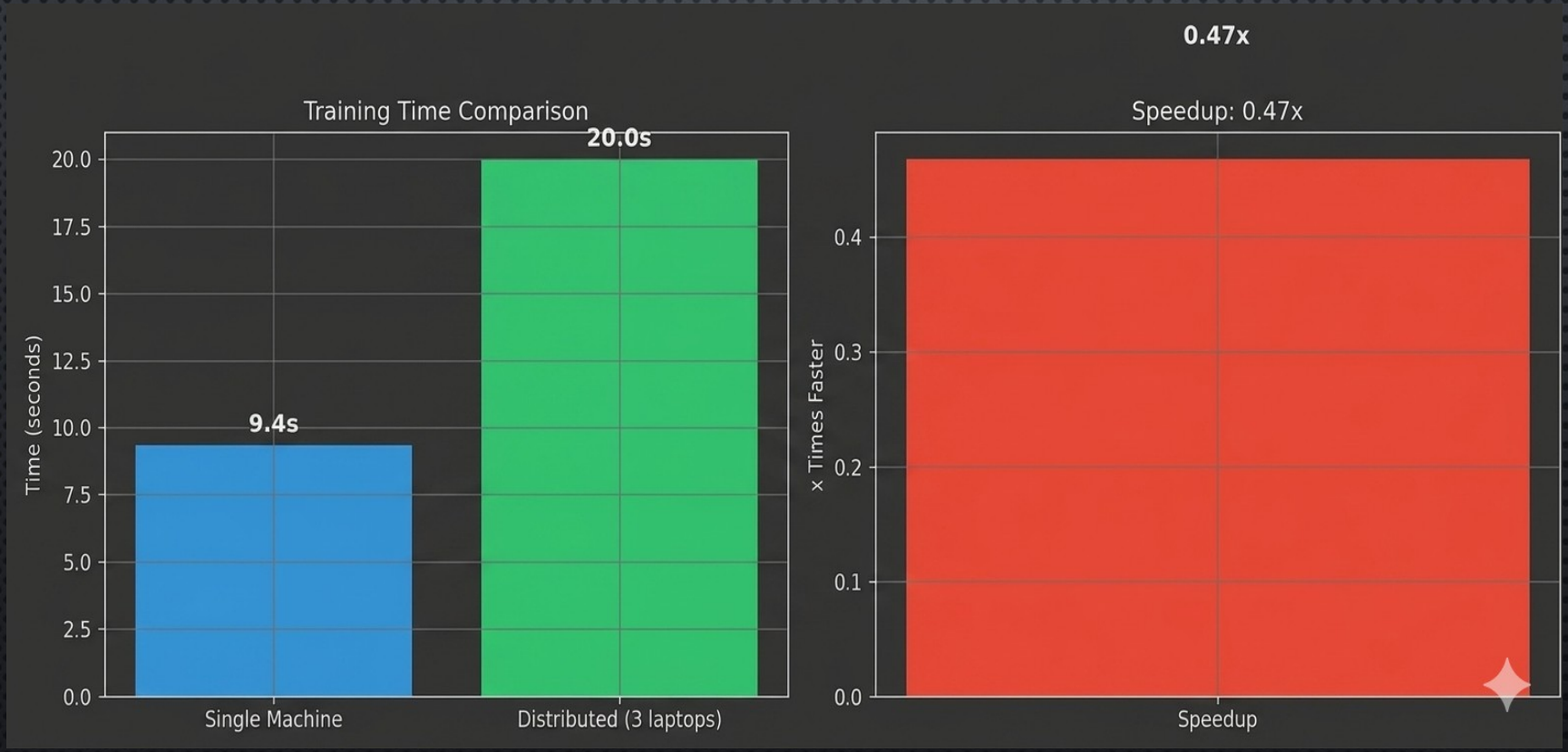
Performance Comparison

- Single Machine Baseline: 9.4 seconds.
- Distributed Cluster:20.0 seconds.
- Accuracy: Both methods converged to similar cluster centers.
- Observation: Distributed time includes network overhead (serialization, latency, 200+ round trips).
- LESSON:FROM WHAT WE SAW DISTRIBUTED IS MOST EFFECTIVE FOR COMPLEX MODELS HENCE ON IRIS MODEL SINGLE MACHINE DOMINATES

SINGLE MACHINE K-MEANS CLUSTERING



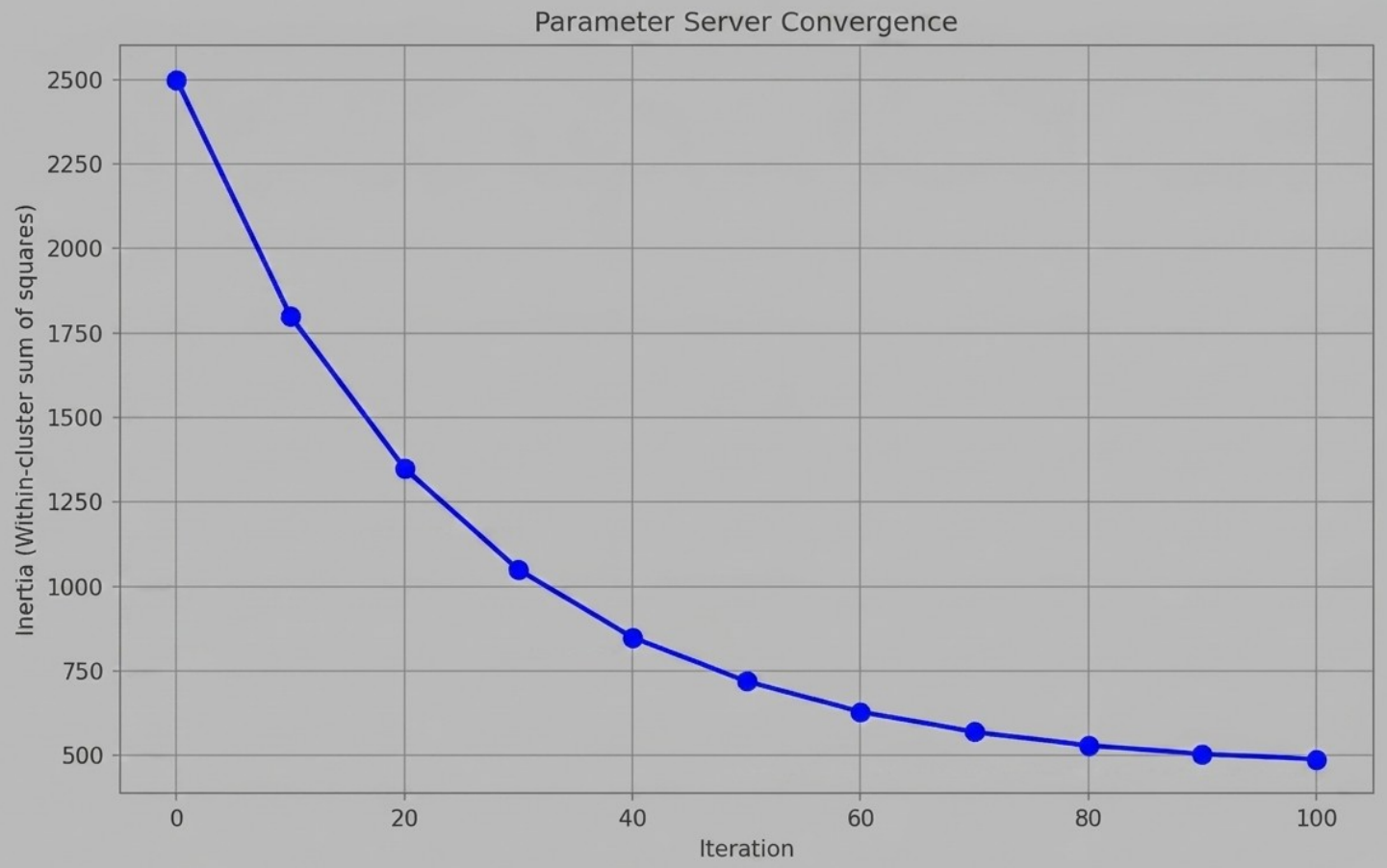
PERFORMANCE COMPARISON DIAGRAM



Accuracy using convergence analysis

- Metric: Inertia (Sum of squared distances to centers).
- Outcome: Both methods converged to similar cluster centers.
- Result: The distributed system successfully learned the data patterns despite network delays.

ACCURACY DIAGRAM



Challenges & Solutions

- Challenge: Communication Overhead. Network latency outweighed simple compute time.
- Solution: Implemented async PUB/SUB pattern with ZeroMQ to reduce blocking.
- Challenge: Node Synchronization. Slow nodes can bottleneck the server.
- Solution: Implemented 5-second timeout with retry logic. Server continues if a node fails.
- Challenge: ZMQ Deadlocks. Original blocking pattern hung on dropped packets.
- Solution: Switched to PUSH/PULL and PUB/SUB patterns for reliability.

Reflection & Lessons

- Key Lesson: Amdahl's Law. Distributed training shines when Compute >> Communication.
- Real-World: Effective for Deep Learning (hours/days training), but overhead-heavy for simple models.
- Future Work: Scale to 10+ nodes, implement fault tolerance with Spark or Ray.
- Takeaway: Successfully built a working distributed parameter server from scratch.

Live demo

- 1. Baseline Run: `single_machine_kmeans.py` (9.4s)
- 2. Distributed Run: `server.py` + 2x `worker.py` (20.0s)
- 3. Show: Data partitioning logs, convergence updates, and final inertia.
- 4. Node Failure: Demonstrate server timeout recovery.

Conclusion

- Success: Built a fully functional distributed ML system from scratch.
- Real-World Application: This architecture scales to handle Big Data and Deep Learning workloads.
- Future Work:
- Implement automatic fault tolerance.
- Scale to larger clusters (10+ nodes).